⚡ ZERO-DOWNTIME MIGRATION

Migrating 200+ Legacy Tests Without Freezing Releases

Legacy Selenium and Cypress suites eventually become a SaaS team's biggest deployment bottleneck.

Here is the **Dual-Run CI Blueprint** we used to migrate 200+ tests to TypeScript Playwright—cutting CI runtime by **30%** with zero release downtime.

🎯 **Who This Is For**

Engineering Leaders and Platform Teams stuck with slow, brittle E2E suites who cannot afford a 4-week feature freeze.

AV

Alberto E. Vilano Reyna

Senior SDET & AI Quality Engineer • avilano.dev

Swipe ➔

WHY MOST MIGRATIONS STALL

The "Big Bang" Rewrite Anti-Pattern

Teams try to freeze feature work and rewrite everything at once—or copy-paste brittle patterns into a new framework.

🛑 **1. Feature Freezes Always Fail**

Product teams keep shipping while the rewrite lags behind, creating permanent merge conflicts and stale coverage.

🐌 **2. Repeated UI Login Overhead**

Legacy suites spend up to **40% of total CI time** typing credentials into login forms before every single test.

🧩 **3. Porting Brittle XPaths**

Moving CSS/XPath selectors into Playwright just gives you faster-failing flaky tests.

AV

Alberto E. Vilano Reyna

avilano.dev

Swipe ➔

Why Cutovers Break Developer Trust

Flipping the merge gate to an unproven test suite destroys engineering velocity:

⚠️ **False Positives Block Good PRs**

Without parity verification against the legacy suite, one misconfigured wait stalls every deployment.

🎨 **Tailwind & CSS Refactors Break Selectors**

Procedural scripts tied to class names force QA to rewrite tests every time frontend styling changes.

AV

Alberto E. Vilano Reyna
avilano.dev

Swipe ➔

NEVER STOP THE LINE

3-Phase Dual-Run Migration Architecture

Run old and new suites side by side per domain, and cut over only after proving speed and parity.

1️⃣ **Phase 1: Dual-Run in GitHub Actions**

Legacy suite stays the required merge gate while migrated Playwright specs run in parallel as informational checks.

2️⃣ **Phase 2: Parity & Speed Verification**

Verify each migrated domain runs in under 1/3rd the legacy time with zero false positives.

3️⃣ **Phase 3: Domain-by-Domain Cutover**

Delete legacy specs per module and promote Playwright to the required PR check—zero release freeze.

AV

Alberto E. Vilano Reyna

avilano.dev

Accessible POMs & Sub-50ms Auth Seeding

Replace brittle XPaths and slow UI logins with Accessibility Tree locators and cached API sessions.

❌ LEGACY: XPATH + UI LOGIN ON EVERY TEST

```
// 40% of runtime spent logging in via UI
await page.locator('//div[@class="btn-save"]').click();
```

✅ PLAYWRIGHT: ROLE LOCATORS + STORAGESTATE

```
// auth.setup.ts caches JWT once -> <50ms per test
this.submitButton = page.getByRole('button', { name: /save changes/i });
this.searchField  = page.getByLabel(/search courses/i);
```

AV

Alberto E. Vilano Reyna
avilano.dev

200+ Tests Migrated: Measured Outcomes

Production impact across a multi-tenant enterprise SaaS CI pipeline:

METRIC	OUTCOME
Migration Scope	**200+** E2E tests moved to TypeScript Playwright
CI Pipeline Runtime	**-30%** overall workflow duration
Auth Boot Overhead	**< 50ms** via cached `storageState`
Test Maintenance	**> 60% reduction** via role-based POMs

AV

Alberto E. Vilano Reyna
avilano.dev

3 Rules for a Zero-Downtime Playwright Migration

Modernize your E2E suite without halting product delivery:

- ✓ **Run Dual-Run CI:** Keep legacy as the merge gate until Playwright proves parity.
- ✓ **Seed auth via API (`storageState`):** Eliminate UI login overhead on every spec.
- ✓ **Query the Accessibility Tree:** Use `getByRole()` and `getByLabel()` inside Page Objects.

Read the Full Migration Blueprint

Full Dual-Run CI setup and Page Object patterns at **avilano.dev** (link in first comment).

AV
Alberto E. Vilano Reyna
Follow for Playwright & CI Architecture • avilano.dev

Save & Follow ★